WorkCore API Integration Guide

Version: 1.4 Date: March 9, 2026 Primary API URL: `https://api.hq21.tech` Secondary API URL: `https://api.runwcr.com`

1. Purpose

This document is a practical integration guide for backend and platform teams that need to integrate with the WorkCore runtime API.

Source-of-truth contract:

- OpenAPI: `https://api.hq21.tech/openapi.yaml`
- API reference: `https://api.hq21.tech/api-reference`
- Agent integration entrypoint: `https://api.hq21.tech/agent-integration-kit`

2. Authentication model

WorkCore API is protected with bearer token authentication.

Send on protected endpoints:

- `Authorization: Bearer <WORKCORE_API_AUTH_TOKEN>`

No bearer token is required for:

- `GET /health`
- `GET /openapi.yaml`
- `GET /api-reference`
- `GET /workflow-authoring-guide`
- `GET /agent-integration-kit`
- `GET /agent-integration-kit.json`
- `GET /agent-integration-test`
- `GET /agent-integration-test.json`
- `POST /agent-integration-test/validate-draft`
- `GET /schemas/*`
- `POST /webhooks/inbound/{integration_key}` (signature-based)

Bearer auth is required for:

- `GET /agent-integration-logs`

If you are an internal platform operator and have Azure access, you can read the current API token from Key Vault:

```
az keyvault secret show \
  --vault-name kv-workcore-prod-uaen \
```

```
  --name workcore-api-auth-token \
  --query value -o tsv
```

3. Required headers

Common headers

- `Authorization: Bearer <token>` (required on protected endpoints)
- `X-Tenant-Id: <tenant>` (recommended on all calls, required for strict multi-tenant paths such as `/chat`)
- `X-Correlation-Id: <id>` (recommended)
- `X-Trace-Id: <id>` (recommended)

Workflow authoring/run headers

- `X-Project-Id: <project_id>` is required for `/workflows*` APIs.

Retry safety

- `Idempotency-Key: <key>` is strongly recommended for mutating requests (`POST/PATCH/DELETE`).

4. Quick start

```
export BASE_URL="https://api.hq21.tech"
export TOKEN="<WORKCORE_API_AUTH_TOKEN>"
export TENANT_ID="local"
export PROJECT_ID="proj_hq21"
```

Health check:

```
curl -sS "$BASE_URL/health"
# expected: ok
```

List projects (protected endpoint):

```
curl -sS "$BASE_URL/projects" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID"
```

List workflows (requires project scope):

```
curl -sS "$BASE_URL/workflows" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID"
```

5. Minimal integration flow

5.1 Create project

```
curl -sS -X POST "$BASE_URL/projects" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "Content-Type: application/json" \
  -d '{
    "project_id": "proj_hq21",
    "project_name": "HQ21 Production"
  }'
```

5.2 Create workflow draft

```
curl -sS -X POST "$BASE_URL/workflows" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Order triage",
    "description": "Classify customer intent and route",
    "draft": {
      "nodes": [
        {"id": "start", "type": "start"},
        {"id": "end", "type": "end"}
      ],
      "edges": [
        {"source": "start", "target": "end"}
      ]
    }
  }'
```

5.3 Publish workflow

```
curl -sS -X POST "$BASE_URL/workflows/<workflow_id>/publish" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID"
```

5.4 Start run

```
curl -sS -X POST "$BASE_URL/workflows/<workflow_id>/runs" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID" \
  -H "Idempotency-Key: run-$(date +%s)" \
  -H "Content-Type: application/json" \
  -d '{
    "inputs": {
      "customer_text": "I need help with my order"
    }
  }'
```

5.5 Poll run state

```
curl -sS "$BASE_URL/runs/<run_id>" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID"
```

6. SSE streaming and reconnect

Run progress stream:

```
curl -N "$BASE_URL/runs/<run_id>/stream" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID" \
  -H "Accept: text/event-stream"
```

Reconnect from last event:

```
curl -N "$BASE_URL/runs/<run_id>/stream" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID" \
  -H "Last-Event-ID: evt_abc123"
```

SSE events include `id`, `event`, and JSON `data` with:

- `event_id`, `sequence`, `timestamp`
- `run_id`, `workflow_id`, `version_id`, `node_id`
- `type`, `payload`
- `correlation_id`, `trace_id`, `tenant_id`, `project_id`, `import_run_id`

7. Webhooks

7.1 Inbound webhook (to WorkCore)

Endpoint:

- `POST /webhooks/inbound/{integration_key}`

Headers:

- `X-Webhook-Timestamp: <unix_ts>`
- `X-Webhook-Signature: t=<unix_ts>,v1=<hmac_sha256>`
- optional `Idempotency-Key: <key>`

Signature payload:

- `"<timestamp>.<raw_request_body>"`

Signature example:

```
SECRET="<inbound_secret>"
TS="$(date +%s)"
BODY='{"action":"start_run","workflow_id":"wf_123","inputs":{"customer_text":"Hello"}}'
SIG="$(printf "%s.%s" "$TS" "$BODY" | openssl dgst -sha256 -hmac "$SECRET" -hex | sed
's/^.* //')"

curl -sS -X POST "$BASE_URL/webhooks/inbound/default" \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Timestamp: $TS" \
  -H "X-Webhook-Signature: t=$TS,v1=$SIG" \
  -H "Idempotency-Key: wh-$(date +%s)" \
  -d "$BODY"
```

Supported actions:

- `start_run`
- `resume_interrupt`

7.2 Outbound webhooks (from WorkCore)

Register subscription:

```
curl -sS -X POST "$BASE_URL/webhooks/outbound" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://your-service.example.com/workcore/events",
    "event_types": ["run_completed", "run_failed", "interrupt_created", "node_failed"]
  }'
```

List subscriptions:

```
curl -sS "$BASE_URL/webhooks/outbound" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID"
```

Delete subscription:

```
curl -sS -X DELETE "$BASE_URL/webhooks/outbound/<subscription_id>" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID"
```

8. Chat endpoint

Public chat endpoint:

- `POST /chat`

Requirements:

- `X-Tenant-Id` is required
- send `Authorization: Bearer <WORKCORE_API_AUTH_TOKEN>` on protected public API calls

For `threads.create`, include `metadata.workflow_id` (required).

9. Error model

All API errors are returned in a standard envelope:

```
{
  "error": {
    "code": "INVALID_ARGUMENT",
    "message": "..."
  },
  "correlation_id": "corr_123"
}
```

Common integration errors:

- `401 UNAUTHORIZED`: missing or invalid bearer token
- `422 ERR_PROJECT_ID_REQUIRED`: missing `X-Project-Id` on workflow endpoints
- `400 INVALID_ARGUMENT`: invalid request body/parameters

10. Production checklist for external teams

1. Token rotation process is defined and tested.
2. `X-Tenant-Id`, `X-Correlation-Id`, `X-Trace-Id` are generated and propagated end-to-end.
3. `X-Project-Id` is always set for `/workflows*` APIs.
4. All mutating calls send stable `Idempotency-Key`.
5. SSE consumers support reconnect with `Last-Event-ID`.

6. Webhook consumers verify signatures and handle retries idempotently.
7. Monitoring dashboards alert on 401/429/5xx spikes.

11. Operational contacts

- API contract and behavior changes must be validated against `openapi.yaml`.
- For environment-specific token/domain issues, contact the WorkCore platform owner.